Ders 5 Genişletilmiş Notu: Yazılım Ölçümü: Halstead, McCabe, Güvenirlik ve CK Metrikleri

Ders 5 Genişletilmiş Notu: Yazılım Ölçümü: Halstead, McCabe, Güvenirlik ve CK Metrikleri

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Planı
- 2. V Model: Doğrulama ve Geçerleme
- 3. Birim Testi ve Test Senaryosu
- 4. Bileşen, Bütünlük ve Regresyon Testi
- 5. Sistem Testi ve Kalite Özellikleri
- 6. Onaylama ve Kabul Testi
- 7. Performans Testi
- 8. Test Tipleri: Saydam Kutu ve Kara Kutu
- 9. Hata Ayıklama ve Hata Düzeyleri
- 10. Yazılım Ölçümü Neden Yapılır?
- 11. Halstead Metriği
- 12. McCabe Döngüsel Karmaşıklık Ölçüsü
- 13. Büyüklük Ölçütleri: Satır Sayısı ve İşlev Noktası
- 14. Yazılım Güvenirliği ve Güvenirlik Modelleri
- 15. Yazılım Güvenirliğini Sağlama Adımları
- 16. CK Metrikleri
- 17. Gereksinim Belgesi (SRS) ve IEEE 830
- Örnek Sorular

1. Dersin Planı

Hoca, önceki haftalarda işlenen yazılım kalite özellikleri, kalite aktiviteleri, yazılım standartları, gözden geçirme ve inceleme konularını artık tekrar etmeyeceğini söyledi. Bu derste önce test aşamasından itibaren görülenler gözden geçirildi. Ardından yeni konu olarak yazılım ölçümü ve değerlendirmesi işlendi. Hoca test kavramı önemli olduğu için tekrarı biraz uzun tuttuğunu dersin ortasında belirtti.

2. V Model: Doğrulama ve Geçerleme

Test sürecinde ilk işlenen konu, test sürecinin adımları ve bu adımların yazılım geliştirme süreciyle ilişkisiydi. Bu ilişki V model yaklaşımı üzerinden anlatılır. Hoca daha önce slayttaki geçerleme ve doğrulama kelimelerini düzelttiğini, ayrıca regresyon, kabul testi ve beta testi gibi ek süreç adımlarını şekle eklediğini hatırlattı. Terimler şöyle kullanılır: doğrulama (verification), geçerleme (validation).

2.1 Doğrulama (Verification)

Hoca bir öğrenciye "Verification neydi?" diye sordu. Öğrencinin cevabı "doğru yolla oluşturma" idi. Hoca bunu açtı: V modelin sol tarafında yazılım geliştirme süreci adımları vardır. Doğrulama, geliştirme sürecinde doğru yolu izleyip izlemediğimizin kontrolüdür.

- Kodlamadan hemen sonra, kodlanan birimler üzerinde birim testi yapılır. Burada "bu algoritma doğru yolla oluşturulmuş mu, doğru yolda mıyız?" diye bakılır.
- Bütünlük testinde, gereksinim analizindeki işlemlerin tasarıma doğru yolla aktarılıp aktarılmadığına bakılır. Bu kontrol eldeki belgeler izlenerek ve yazılım çalıştırılarak yapılır. Bu da doğrulamadır.

2.2 Geçerleme (Validation)

Geçerlemede, ürettiğimiz yazılım ürününün istediğimiz ürün olup olmadığı kontrol edilir. Müşterinin istediği işlevler gerçekleştirdiğimiz yazılımda var mı? Müşterinin istediği ürün spesifikasyonu sistemimizde sağlanıyor mu? Bu sorulara cevap aranır.

2.3 Test Adımlarının Kullandığı Belgeler

V modelde test süreci adımları, geliştirme sürecinde hazırlanan belgeleri kullanır:

- Yüksek düzey tasarım ile bütünlük testi arasında tasarım belgesinden faydalanılır.
- Sistem testi gereksinim belgesinden faydalanarak yapılır.
- Müşteriden alınan bilgiler ve ürün istek bildirimleri, işlemsel kullanım sırasında kontrol edilir.

Hoca, V modelin bu sırayla, yani geliştirme sürecine ters sırada yürünerek okunduğunu vurguladı: birim testi, bütünlük testi, sistem testi ve işlemsel kullanıma geçiş. "Size bir yazılım verildi, bunu hangi sırada test edersiniz?" diye sorulduğunda beklenen cevap bu test süreci adımlarıdır.

3. Birim Testi ve Test Senaryosu

Birim testinde yazılım tasarımının en küçük birimi olan modül üzerinde, ayrıntılı tasarım tanımlarına dayanarak hatalar bulunmaya çalışılır. Bu bir doğrulama sürecidir ve genellikle saydam kutu testi uygulanır. Modüller, ara birimler ve yerel veri yapıları test edilir.

Hoca birim testi şeklini hatırlattı. Test edilen birim her zaman doğrudan test verisi gönderilebilecek bir birim olmayabilir. Veri girişi, onu çağıran modül tarafından yapılabilir; bu görevi test sürücüsü (test driver) üstlenir. Test edilen birimin çağırdığı alt programların yerine yapay modüller, yani kukla modüller (stub) konur. Sonuçlar yine test sürücüsü aracılığıyla aktarılır.

3.1 Test Senaryosu İki Düzeyde

Birim testinde test senaryoları (test case) önemlidir. Hoca test senaryosu kavramını iki düzeyde anlattı.

Birinci düzey kullanıcı testidir. Geçerleme yapılırken özel bir gereksinim senaryosu oluşturulur ve bu senaryonun gereksinim spesifikasyonuna uygun olup olmadığına bakılır. Hocanın örneği öğrenci bilgi sistemiydi:

- Dersi alan öğrencilerden çift anadal yapanların durumunu sorgulamak veya ders kayıtlarını yapmak bir test senaryosudur.
- Mühendislik tamamlama öğrencilerinin ders kayıtlarını oluşturmak başka bir senaryodur.
- Böyle farklı senaryolar oluşturup her kullanıcı grubu için sistemin doğru çalıştığı test edilir.

İkinci düzey birim testidir. Burada test senaryosu daha mikro seviyededir: En alttaki modülün bile hangi girdiyi aldığında hangi çıktıyı üretmesi gerektiği tanımlanır. Her modül kendine düşen küçük alt işlemden sorumludur. Bu iş içeriden bakılarak, yani saydam kutu testi biçiminde,

modülün iç yapısına ve algoritmasına bakılarak test edilir. Algoritmadaki bütün yolların denenmesi istenir. Bunun için de bütün yolları deneyebilecek test verisi sağlanmalıdır. Hoca bunu "Test yapacağız ama hangi veriyle test yapacağız?" sorusuyla özetledi.

4. Bileşen, Bütünlük ve Regresyon Testi

Hoca test süreci şeklini, kaçıranlar için yeniden gösterdi: birim testi, bileşen testi, bütünlük testi. Bileşen testi üzerinde ayrıca durmadığını söyledi. Bileşen (component) mimarisi kullanılmışsa, birim testi yapılmış birimlerden oluşan bileşenler de test edilir.

4.1 Bütünlük Testi

Bütünlük testinde, modüllerin nasıl birleştirildiği ve yazılımın tamamı test edilir. Çıkış noktası şudur: Modüller bağımsız değildir. Birbirleriyle bağlantılıdırlar ve bu bağlantılar bir yazılım ara birimi tarafından sağlanır. Birleştirme sırasında şu sorunlar çıkabilir:

- Modüllerde tek tek hata görülmese bile, birleştirilince hata toleransları eklenerek büyüyebilir.
- Genel (ortak) veri yapılarında sorun olabilir.
- Her birim kendine düşeni yapsa bile, bütünleştirilince istenen iş gerçekleşmeyebilir.

Bütün yazılımı birleştirip tek seferde bütün olarak test etmek verimli bir yol değildir. En iyisi artırmalı sınamadır. Artırmalı sınama yukarıdan aşağıya ya da aşağıdan yukarıya yapılır.

- **Yukarıdan aşağıya:** Yazılıma bütün olarak bakılır, ama alttaki modüller ilk başta birer yapay modül (stub) gibi düşünülür. Aşağı indikçe bunlar gerçek modülleriyle değiştirilir.
- **Aşağıdan yukarıya:** Önce en alttaki modüllerin tamamlanması gerekir. Örneğin Modül 7'nin çağırdığı Modül 1 yerine yerleştirilerek bir bütünlük testi yapılır. Sonra Modül 8'in çağırdığı alt programların hepsinin test edilip edilmediğine bakılır ve bunlar yerleştirilir. Modül 9 için de aynısı yapılır. Böylece alttan yukarı çıkılır. Burada bir paralellik vardır: Modül 7, Modül 8 ve Modül 9 kendi alt modülleriyle aynı anda birleştirilip test edilebilir.

4.2 Regresyon Testi

Test edilmiş bir programda sonradan değişiklik yapılmışsa ilgili birimler yeniden test edilir. Uygulama ortamında gerekli değişiklikler yapıldıktan sonra yapılan bu testlere regresyon testi denir. Yeni sürümlerde de regresyon testi yapılabilir. Regresyon testinde yardımcı test araçlarından (test tool) faydalanılabilir. Bu araçlarla daha önce test edilmiş bölümlerin testleri yeniden elde edilebilir.

5. Sistem Testi ve Kalite Özellikleri

Bütünlük testinden sonra (araya eklemeler ve tamamlamalar için regresyon testi girer) sistem testi yapılır. Bütünlük testinde yazılımın doğru çalıştığı anlaşılmış olur. Sistem testinde ise yazılımın sistem donanımıyla uyumlu çalıştığı onaylanır. Ayrıca spesifikasyonlara uygunluk kontrol edilir: Kalite özelliklerinden bu yazılım için özellikle istenmiş olanlara bakılır.

Uygulanabilecek sistem testleri:

- **Güvenlik testi:** Sistem zararlı dış müdahalelerden korunabiliyor mu?
- **Dayanıklılık (stres) testi:** Sistem aşırı yüklendiğinde buna dayanıklı mı?
- **Yetenek (performans) testi:** Yazılımın işlem süresi ve yeteneği baştaki isteklere uygun mu? Daha önce belirtilmiş cevap süresi ve geri dönüş süresi gibi değerler isteklere uygun mu?

Böylece sistem testi kalite özellikleriyle birleştirilmiş olur.

5.1 McCall Hatırlatması ve Soru-Cevap

Hoca kalite özellikleriyle ilgili bir hatırlatma yaptı ve bir öğrenciye sordu.

Soru: Yazılım kalite özelliklerine hangi açılardan bakıyorduk? **Cevap:** Öğrenci önce "kalitenin yönetimi, kontrolü, denetimi gibi şeyler mi?" diye sordu. Hoca düzeltti: McCall'a göre yazılım

kalite özelliklerine üç bakış açısıyla bakılır: kullanıcı açısından, geliştirici açısından ve işletici açısından.

Soru: Kullanıcı yazılıma dışarıdan bakar. Yazılımın iyi test edilip edilmediğini hangi özelliklere bakarak anlar? Bunu sistem testiyle ilişkilendirebilir miyiz? **Cevap:** Öğrenci verimlilik, güvenlik ve dayanıklılık (stres) dedi. Hoca bunları onayladı.

Soru: Dayanıklılıktan kastettiğimiz nedir? **Cevap:** Öğrencinin cevabı: Büyük bir program çok kullanıldığında aynı verimi verebilmesidir. Örneğin kullanıcı sayısı belirlenen sınırın üzerine çıktığında, ya da sınırdaki sayılarda sistem aynı verimlilikle çalışıyor mu? Hoca onayladı.

Hocanın vardığı sonuç şudur: Sistem testiyle fonksiyonel olmayan yazılım kalite özellikleri eşleştirilir. Sistem testinde bu özelliklere bakılmış olur.

> **Ders sonrası not:** McCall modelinin özgün üç grubu ürün işletimi (product operation), ürün gözden geçirme (product revision) ve ürün geçişidir (product transition). Hoca bunları kullanıcı, geliştirici ve işletici bakış açıları olarak anlattı; eşleşme büyük olasılıkla sırasıyla işletim, gözden geçirme ve geçiş gruplarına denk gelir.

6. Onaylama ve Kabul Testi

Bütünlük testinden ve sistem testinden sonra onaylama testi yapılır. Burada yazılımın müşteri ve kullanıcı beklentilerini karşılayıp karşılamadığına bakılır. Onaylama testi yazılımdan, gereksinimlerden ve kullanıcı belgelerinden faydalanır:

- Yazılımı kullanıcıya benzer bir grup ya da doğrudan kullanıcı çalıştırır.
- Çalıştırırken kontrol edilen şey gereksinimlerdir: Bütün gereksinimler bu yazılımda sağlanıyor mu?
- Kullanıcı belgelerinden de faydalanılır. Uygulamadaki yardım (help) mekanizması ya da kullanıcıyı yönlendiren kâğıt üzerindeki açıklamalar izlenerek, kullanıcı bütün gereksinimlere ulaşabiliyor mu, buna bakılır.

6.1 Düzen (Konfigürasyon) İncelemesi

Onaylama sırasında yazılımın düzeni, yani konfigürasyonu da incelenir. Hoca bunun için Türkçe “düzen” kelimesini kullandığını söyledi. Yazılımı oluşturan bütün elemanlar (kullanıcı belgeleri, tasarım belgeleri, kayıtlar, test belgeleri) incelenir. İki şeye bakılır:

- Bu elemanlar istenen formatta mı?
- Birbirleriyle ilişkilendirilmiş mi? Örneğin hangi program hangi test verisiyle denendi, hangi tasarım belgesi hangi programı anlatıyor?

6.2 Kabul Testi, Alfa ve Beta Testi

Kullanıcı kabul testi (acceptance test) bu ikisini de içerir: Program, hazırlanan kullanıcı belgeleri ışığında çalıştırıldığında gereksinimleri karşılıyor mu; ve yazılımın bütün elemanları tam ve birbiriyle ilişkilendirilmiş mi? Kabul testinden geçen yazılımı kullanıcı teslim alabilir.

Kabul testini yapabilmek için önce yazılım ekibinde kullanıcıya benzer bir grup oluşturulup yazılım kontrol edilir (alfa testi). Sonra uygulama doğrudan kullanıcıya verilip test etmesi istenir. Bu beta testidir: Sistemin potansiyel kullanıcılara test ettirilmesi.

7. Performans Testi

Aradan sonra hoca sistem testindeki performans testine kısaca döndü. Performans testinde performans seviyeleri belirlenir, belirli bir yük altında sistemin çalışma zamanı performansı ölçülür ve sistem ayarları test edilir. Performans testi şu sorulara cevap verir:

- Sistem gereksinimleri karşılanıyor mu?
- Normal şartlar altında sistem nasıl davranıyor?
- Sistem trafiğinde artış olursa bu durum işlem süresini ve işlevselliği nasıl etkiliyor?
- Kullanıcı seviyeleri arasında performans problemi var mı?

8. Test Tipleri: Saydam Kutu ve Kara Kutu

Geçen hafta görülen iki test tipi tekrarlandı. Kara kutu testinde uygulamaya dışarıdan bakılır, iç yapıyla ilgilenilmez. Saydam kutu testinde (white box testing) ise tamamen yazılımın iç yapısı üzerinde durulur: İç yapı istenen işlemleri gerçekleştiriyor mu, algoritma doğru mu?

Hoca bu iki testin birbirine tercih edilecek şeyler olmadığını, ikisinin de yapılması gerektiğini yeniden söyledi.

8.1 Saydam Kutu Testi

Saydam kutu testinde işlemsel tasarımın kontrol yapısı test edilir. Bütün iş adımları doğru gerçekleştiriliyor mu, algoritmalar doğru yürüyor mu, buna bakılır. Bu testte:

- Algoritmadaki bütün yolları denemek için gereken test verisi üretilir.
- Döngülerin kontrol değerleri test edilir: başlangıç, bitiş ve artış doğru mu?
- Bütün iç veri yapıları kontrol edilir.

Saydam kutu testine iki örnek görülmüştü: temel yol testi (basis path testing) ve döngü testi.

Temel yol testi: İşlemsel tasarımın mantıksal karmaşıklığını ölçmeye ve bu ölçüye göre uygulama yolları için bir temel grup oluşturmaya dayanır. Test sırasında programdaki her deyim en az bir kez çalıştırılır. Adımlar:

1. Ayrıntılı tasarımdaki ya da kaynak programdaki yapıya (algoritmaya) göre bir akış grafı çizilir.
2. Akış grafı üzerinde McCabe döngüsel karmaşıklık ölçüsü hesaplanır. Düğümler ve yollar belirlenip kaç temel yol olduğu bulunur.
3. Her yol için bir test programı düzenlenir. Yani algoritmanın o yoluna gitmesini sağlayacak bir giriş veri grubu oluşturulur.
4. Her test programı uygulanır ve sonuç beklenen sonuçla karşılaştırılır.

Hoca döngüsel karmaşıklığın nasıl hesaplanacağını ve akış grafının nasıl çizileceğini ölçme konusunda göreceklerini söyledi (Bölüm 12).

Döngü testi: Döngülere bakılır. Girdisi, çıktısı ve işlemleri her döngüde doğru anlatılmış mı? Hedef şunları bulmaktır: döngü içinde başlama hatası, sonsuz döngü ihtimali ve döngünün sınır hatası. Böylece döngü yapısının geçerliliği onaylanır.

8.2 Kara Kutu Testi

Kara kutu testi, yazılımın bütünleştirilmesi sırasında yazılım ara birimi yapısına da uygulanır. Bu testte yazılımın işlevleri yerine getiriliyor mu, kullanıcının girdiği veri istediği çıktıya ulaşıyor mu, buna bakılır. Mantıksal iç yapıdan çok temel sistem modeli denenir. Genel olarak işlevsel gereksinimler denenir: Kullanıcı istediği işi yapabiliyor mu? Bunun için uygun girdiler girilmelidir. Kara kutu testiyle bulunan hatalar:

- Hatalı ve eksik işlemler.
- Veri yapılarında ve veritabanı erişiminde hatalar.
- Performans hataları.

Kara kutu test tipleri arasında veri onaylama testi ve neden-sonuç grafı çizimi de vardır. Hoca derste iki tanesi üzerinde durdu:

- **Eş değerli bölümleme (equivalence partitioning):** Programın girdi alanı, test programlarının oluşturulabileceği veri sınıflarına bölünür. Her test programı belirli bir sınıftaki hataları ortaya çıkarır. Böylece daha az test programıyla yetinilir. Yani veri uzayı alt sınıflara bölünür ve her sınıftan bir örnekle uygulama çalıştırılır. Ayrıca geçersiz girdi de verilir. Hoca bunun bütün kara kutu testleri için geçerli olduğunu söyledi: Geçersiz girdiye yanıtın ne olacağı da görülmelidir.
- **Sınır değer analizi (boundary value analysis):** “Hatalar genel olarak nerede yoğunlaşır?” sorusundan yola çıkar. Genel kanı, hataların sınırlarda oluştuğu, ortadaki verinin genellikle daha hatasız olduğudur. Örneğin kullanıcı sayısının maksimum seviyesine yakın bir sayıyla çalıştırıp programda bir çökme olup olmadığına bakmak daha doğrudur.

Yani “hatalar merkezden çok kenarlarda toplanır” varsayımıyla ilerlenir. Bu analiz girdi alanına, bazen de çıktı alanına uygulanır.

Hoca bir de keşif testinden söz etti: Ürün spesifikasyonu ya da gereksinim belgesi olmadan yazılımı hemen genel olarak çalıştırıp hataları bulmaya dayanan dinamik bir kara kutu testidir.

> **Ders sonrası not:** Hoca “keşif testi, smoke test türü” dedi. Bu ikisi farklı kavramlardır. Keşif testi (exploratory testing), belgeye dayanmadan yazılımı kullanarak hata aramaktır; hocanın tarifi buna uyar. Duman testi (smoke test) ise yeni bir sürümün temel işlevlerinin çalışıp çalışmadığına bakan kısa bir ön testtir.

9. Hata Ayıklama ve Hata Düzeyleri

Geçen hafta hata giderme (debug) işlemi üzerinde de durulmuştu. Hata giderme, bulunan hataların düzeltilmesidir. Bulunan hatalara bakıldığında hataların (bug) nedenleri arasında gereksinimler en büyük paya sahipti. Hoca bunun çevik yazılım geliştirmede müşteriyi işin içine katarak biraz azaldığını hatırlattı.

Hata düzeyleri ise “ne zaman teste devam edelim, hangi hatalarda testi tamamen durduralım?” sorusuna cevap verir. Ölümcül hatalarda ve büyük hatalarda genellikle teste devam edilmez. Bazı ekipler büyük hatalarda teste devam edip yazılımı teslim etmemek gibi aralıklar belirler. Bu sınıflama her kurumun kendi yapısına göre baştan belirlenir. Hata sınıflamasını her kurumda altı grupta yapmak gibi bir zorunluluk yoktur.

> **Ders sonrası not:** Kısa nota göre ders 4’teki slaytta kritik ve büyük hatalarda testin devam ettiği, ama yazılımın düzeltilmeden teslim edilmediği yazıyordu. Bu derste hoca ölümcül ve büyük hatalarda genellikle teste devam edilmediğini söyledi. Hocanın da belirttiği gibi bu sınır kuruma göre değişir.

10. Yazılım Ölçümü Neden Yapılır?

Yeni konu yazılım ölçümü ve değerlendirmesiydi (software metrics). Hoca konunun planını şöyle verdi: yazılım ölçümü için kullanılan belli başlı ölçütler, temel yol testinde kullanılan McCabe ölçüsünün nasıl hesaplandığı, güvenirlik kavramı ve güvenirlik modelleri, son olarak da nesneye yönelik yazılımlar için öne çıkan ölçütler.

Tom DeMarco, yazılım üzerinde ölçme ve yazılım mühendisliği konusunda çalışmış biridir. Onun klişeleşmiş cümlesi şudur: Ölçme yapılamazsa kontrol edilemez. Yazılıma uygulandığında: Yazılım üretimini ölçemezsek onu kontrol edemeyiz. “Ölçemediğinizi kontrol edemezsiniz.”

> **Hocanın vurgusu:** Ölçemediğinizi kontrol edemezsiniz (Tom DeMarco).

Ölçmeye ihtiyaç, yöneticiler ve yazılım mühendisleri açısından farklıdır.

10.1 Yöneticiler Neden Ölçer?

- Yeni bir yazılım üretiminin maliyetine karar vermek için bitmiş yazılımları ölçerler.
- Bölüm çalışanlarının ne kadar iş yaptığını ve ne kadar iş gücü harcadığını belirlemek, ücretlerini belirlemek için personel üretkenliğini ölçerler.
- Farklı projelerde geliştirilen yazılım ürünlerini kıyaslamak için kalitelerini ve büyüklüklerini ölçerler.
- Projeler için ölçülebilir hedefler koymak isterler. Örneğin yazılım sisteminin ne kadar güvenilir olduğunu belirlemek için ölçme sistemlerine ya da ölçme modellerine ihtiyaç vardır.
- Şirkette kullanılmaya başlanacak yöntemlerin ve yazılım geliştirme araçlarının etkinliğini saptamak için ölçerler.

10.2 Teknik Kişiler Neden Ölçer?

- Bitirilen ürünün kalitesi ve performansıyla ilgili gereksinimleri daha sonra kolay belirlemek için.
- Sistemin kalitesini daha kolay izleyebilmek için.

- Tasarım süresince yapılan değişiklikleri ve gözden geçirme ya da test adımlarında bulunan hataları değerlendirmek için süreç ölçümleri yaparlar.
- Var olan ürünlerden ve uygulanan süreçlerden yararlanıp sonraki ürünlerde izleyecekleri yönteme karar vermek için.

Hocaya göre bu ölçmeler, tasarımda doğru karar vermek için bir arka plan oluşturur. Örneğin bitmiş yazılımlarda ölçüm yapılmışsa, buradan çıkan sonuçla yeni başlanacak nesneye yönelik bir yazılımda bağımlılığın (coupling) ve uyumun (cohesion) ne ölçüde olması gerektiğine doğru karar verilebilir.

11. Halstead Metriği

Kaynak programda büyüklük ve karmaşıklık ölçmek için bazı ölçütler vardır. Amaç programın aritmetiksel yapısı hakkında fikir vermek ve bitmiş yazılımları kıyaslayabilmektir. Kaynak programın hesaplama yapısının karmaşıklığını belirtmek için Halstead metriği kullanılır. Çok eski yıllardan beri kullanılan bir ölçüttür. Hoca bu metriği öğrencilerin başka derslerde de görmüş olacağını söyledi.

Halstead metriğinde sayılan değerler işleç (operator) sayısı, işlenen (operand) sayısı ve bunların yinelenmelerinin toplamıdır. Bunlara bağlı olarak programın uzunluğu, programın hacmi ve programın gücü gibi değerler hesaplanır. Böylece aritmetiksel hesaplama bakımından yazılımın karmaşıklığı ölçülür.

Kısa nottaki slayt gösterimiyle sayılan değerler şunlardır:

- m: programdaki farklı işleç sayısı.
- k: programdaki farklı işlenen sayısı.
- M: işleçlerin toplam yinelenme sayısı.
- K: işlenenlerin toplam yinelenme sayısı.

Hesaplanan ölçütler:

$$U = m \log_2 m + k \log_2 k \qquad \text{(program uzunluğu)}$$

$$H = (M + K)\ \log_2(m + k) \qquad \text{(program hacmi)}$$

$$G = \frac{H}{O} \qquad \text{(program gücü; } O \text{ oransal hacim, } O < 1\text{)}$$

Ders sonrası not: Literatürde bu ölçütler büyük olasılıkla şu adlarla geçer: farklı işleç sayısı n_1 $(= m)$, farklı işlenen sayısı n_2 $(= k)$, toplam işleç sayısı N_1 $(= M)$, toplam işlenen sayısı N_2 $(= K)$. $U = n_1 \log_2 n_1 + n_2 \log_2 n_2$ aslında **tahmini** program uzunluğudur ($\hat{N}$); gerçek uzunluk $N = N_1 + N_2$'dir. H program hacmidir ($V = N \log_2 n$, $n = n_1 + n_2$). "Program gücü" diye çevrilen G ise Halstead'in çaba (effort) ölçüsüdür: $E = V/L$; burada L program düzeyidir ve oransal hacim olarak anılan değer budur. Formüllerin tutarlılığı python ile denetlendi.

Ek bilgi (derste anlatılmadı): Halstead'in zorluk (difficulty) ölçüsü $D = \frac{n_1}{2} \cdot \frac{N_2}{n_2}$ ile tanımlanır ve program düzeyi bunun tersidir, $L = 1/D$. Bu durumda çaba $E = V/L = D \cdot V$ olur. L en iyi durumda 1'e yaklaşır, gerçek programlarda 1'den küçüktür; bu, slayttaki "oransal hacim daima 1'den küçük" ifadesiyle uyumludur. Bazı kaynaklar çabayı 18'e bölerek tahmini programlama süresini (saniye) de verir.

12. McCabe Döngüsel Karmaşıklık Ölçüsü

Döngüsel karmaşıklıkla bir algoritmada kaç yol olduğu bulunur. Hoca bunu saydam kutu testinde, temel yol testinde akış graflarıyla kullandığımızı hatırlattı. McCabe ölçütünde karmaşıklık derecesi, programın kontrol akışı biçiminde gösterilmesine dayanarak belirlenir. Buna

döngüsel karmaşıklık (cyclomatic complexity) denir ve o modülün yazılım karmaşıklığını belirtir.

Program grafı (akış grafı), program akış diyagramından farklıdır. Burada yalnız düğümler ve kenarlar gösterilir. Kontrol akışı içinde her daire, yani her düğüm, bir ya da daha çok kaynak program deyiminden oluşan bir yapıyı temsil eder.

12.1 Hesaplama

Döngüsel karmaşıklığı bulmanın farklı yolları vardır. Birincisi, graf çizildikten sonra graftaki kapalı bölgeleri ve dıştaki bölgeyi saymaktır.

Hoca Pressman'ın Software Engineering kitabından aldığı bir şekil üzerinden gitti: bir ortalama alma yönteminin (procedure average) akış grafı. Burada R_1 dıştaki alandır ve o da sayılır. R_2, R_3, R_4, R_5 ve R_6 kapalı bölgelerdir. Bölgeler sayılınca 6 temel yol olduğu bulunur:

$$V(G) = \text{bölge sayısı} = 1\ (\text{dış}) + 5\ (\text{kapalı}) = 6$$

Aynı değer graftaki düğüm sayısı ve kenar sayısından da hesaplanabilir.

> **Ders sonrası not:** Hoca hesaplamayı "kapalı bölgeler artı en sondaki yarı açık bölge" diye anlattı. Doğru ifade: kapalı bölgeler artı grafın dışında kalan bölge (hocanın örnekte "R_1 dıştaki alan" demesi de buna uyar). Düğüm ve kenar sayısıyla formül $V(G) = E - N + 2$'dir (E kenar, N düğüm sayısı). Pressman'ın bu örneğinde büyük olasılıkla 13 düğüm ve 17 kenar vardır: $17 - 13 + 2 = 6$. Python ile denetlendi; bölge sayımıyla aynı sonuç çıkar.
>
> **Ek bilgi (derste anlatılmadı):** Döngüsel karmaşıklığın üçüncü bir hesaplama yolu $V(G) = P + 1$'dir; P akış grafındaki karar (koşul) düğümü sayısıdır. Pressman'ın ortalama alma örneğinde büyük olasılıkla 5 karar düğümü vardır ve $5 + 1 = 6$ yine aynı sonucu verir. Üç yöntem bağlantılı, düzlemsel bir akış grafında her zaman aynı değeri verir.

12.2 Modül Büyüklüğüne Karar Vermek

McCabe ölçüsü en yüksek modül büyüklüğünü saptamakta da kullanılır. Programı modüllere ayırırken "döngüsel karmaşıklığı 10'dan fazla modül olmasın" gibi bir karar konabilir. Bu durumda karmaşıklığı 10'u geçen modül alt modüllere bölünür. Aksi halde test etmek de zorlaşır.

> **Hocanın vurgusu:** Bir modülün döngüsel karmaşıklığı 10'u geçmemeli; geçerse alt modüllere bölünmeli, yoksa test zorlaşır.

13. Büyüklük Ölçütleri: Satır Sayısı ve İşlev Noktası

Halstead aritmetiksel karmaşıklık için, McCabe döngüsel karmaşıklık (yol sayısı) için kullanılır. Hoca ayrıca, slaytlarda olmayan ama yazılım mühendisliği dersinden hatırlanması gereken büyüklük ölçütlerini sordu. Bir doğrudan ölçüm bir de dolaylı ölçüm olduğunu hatırlattı.

Soru: Yazılım mühendisliği bilgisini kullanarak bir yazılımın büyüklüğünü nasıl ölçeriz? **Cevap:** Öğrenciler satır sayısı ve KLOC dedi. Satır sayısı (lines of code, LOC) satırların sayılmasıdır; KLOC bin kod satırıdır.

Soru: Bunlar doğrudan mı, dolaylı mı ölçümdür? **Cevap:** Doğrudan; satırlar doğrudan sayılır. Dolaylı ölçüm ise işlev noktasıdır (function point).

13.1 İşlev Noktası

İşlev noktasında tek bir değer ölçülmez. Sayılan birden fazla değere bağlı olarak bir değer bulunur.

Soru: İşlev noktasını bulmamıza yardım eden değerler nelerdir? **Cevap:** Hocanın toparladığı değerler:

- Girdi (input) sayısı.
- Çıktı (output) sayısı.
- Dosya sayısı, yani veritabanındaki tablo sayısı.
- Ekran sayısı, yani ara birim sayısı.

Bu değerler işlev noktasını aynı ölçüde etkilemez. Her birinin bir ağırlığı vardır. Örneğin dosya sayısı yazılımın büyüklüğünü daha çok etkiler; girdi ve çıktı sayısının ağırlıkları farklıdır. Bu hesap bir tabloda yapılır.

Bulunan ham işlev noktası değerini düzeltmek için 14 soru sorulur. Bu sorular kalite özellikleriyle ilgilidir: Dağıtık bir uygulama mı? Çevrim içi veri girişi var mı? Yeniden kullanılabilirlik önemli mi? Her sorunun bu yazılımda ne kadar etkin olduğuna göre bir puan verilir. Örneğin uygulama hem veritabanıyla hem uygulamasıyla dağıtıksa 5 verilir. Yeniden kullanılabilirlik için çok değil ama biraz çaba gösterilmişse 1 verilir. Bu puanlar toplanarak bir düzeltme faktörü elde edilir.

Hocaya göre önemli olan, dolaylı ölçümde kullanılan değerleri bilmektir: girdi sayısı, çıktı sayısı, ara birim (ekran) sayısı ve dosya sayısı; bunların yanında kalite özelliklerinden seçilenlerle bulunan bir düzeltme faktörü. Hoca bu konuya daha sonra da geleceklerini söyledi.

> **Ders sonrası not:** Standart işlev noktası yöntemi büyük olasılıkla beş öğe sayar: dış girdiler, dış çıktılar, dış sorgular, iç mantıksal dosyalar ve dış ara birim dosyaları. Düzeltme faktörü $VAF = 0.65 + 0.01 \sum_{i=1}^{14} F_i$ ile bulunur ($F_i \in \{0, \dots, 5\}$), dolayısıyla 0,65 ile 1,35 arasında değişir; işlev noktası $FP = UFP \times VAF$'tır. Aralık python ile denetlendi.

14. Yazılım Güvenirliği ve Güvenirlik Modelleri

Yazılım güvenirliği (reliability) önemli bir kalite özelliğidir. Sonuç doğru olabilir (correctness), ama bu doğruluğa ne kadar güvenebiliriz? Güvenirlik hem kalite özelliklerinde geçer hem de önemli bir ölçme ve değerlendirme kriteridir.

> **Hocanın vurgusu:** Bir yazılımı tümüyle test etmek mümkün değildir. Ticari bir yazılımda bin kod satırında bir hata çıkabilir. Tamamen hatasız yazılım mümkün değildir; amaç mümkün olduğunca çok hatayı yakalamaktır.

Güvenirlik, geçmiş bilgilere ve teste dayalı bir başarı oranıdır:

$$\text{Güvenirlik} = \frac{\text{başarılı işletim süresi}}{\text{toplam işletim süresi}}$$

Yani bir yazılımı ne kadar süredir kullanıyoruz, bunun ne kadarında başarılı olarak kullanıyoruz? Bir de hata sayısı önemlidir: Şu kadar kullanım süresi içinde şu kadar hata çıktı.

> **Ders sonrası not:** "Başarılı süre / toplam süre" oranı literatürde büyük olasılıkla kullanılabilirlik (availability) olarak geçer. Güvenirliğin standart tanımı, yazılımın belirli bir ortamda belirli bir süre boyunca hatasız çalışma olasılığıdır.
>
> **Ek bilgi (derste anlatılmadı):** Güvenirlik ölçümünde sık kullanılan değerler: ortalama hataya kadar geçen süre (MTTF, mean time to failure), ortalama onarım süresi (MTTR, mean time to repair) ve ortalama hatalar arası süre (MTBF, mean time between failures). Aralarındaki ilişki $MTBF = MTTF + MTTR$'dir. Kullanılabilirlik $A = \dfrac{MTTF}{MTTF + MTTR}$ ile bulunur; bu, derste verilen "başarılı süre / toplam süre" oranının karşılığıdır. Hata oranı sabit λ kabul edilirse güvenirlik $R(t) = e^{-\lambda t}$ ve $MTTF = 1/\lambda$ olur. Bu bağıntılar python ile denetlendi.

14.1 Güvenirlik Modelleri

Güvenirlik modelleri istatistikle çok ilişkilidir. Yazılım güvenirliğini hesaplamak için toplam hata sayısı, güvenirlik düzeyi ve sistemde yapılan düzeltmeler sonucunda kalan hata sayısı gibi değerler kullanılır.

Yazılım güvenirliğinin ve geçerliliğinin ölçülmesi 1980'lerden beri üzerinde durulan bir konudur. Bu süre içinde yüze yakın model geliştirilmiştir. Slaytta en yaygın modeller sayıldı:

- Schneidewind modeli.
- Jelinski-Moranda modeli.
- Musa Basic ve Musa-Okumoto logaritmik Poisson yürütme zamanı (execution-time) modeli.
- Littlewood-Verrall modeli.

Bu modellerin ortak özelliği istatistiksel veri kullanmaları ve hata sayısını hesaplamalarıdır: Belirli bir süre çalıştırıldığında ne kadar hata çıktı, hangi düzeltmeden sonra ne kadar hata kaldı?

14.2 Model Seçimi

Hangi modelin iyi olduğuna ve hangisinin kullanılacağına şu sorularla karar verilir:

- Model ile güvenirliği hesaplamak ne kadar kolay?
- Modeli kullanmak için yazılımın tümüne mi, yoksa bir kısmına mı ihtiyaç var?
- Model kolay ve anlaşılır mı?
- Dış etkenlerden etkileniyor mu?
- Bu modele göre yazılım ne kadar test edilmeli ve ne kadar geçerlidir?

15. Yazılım Güvenirliğini Sağlama Adımları

Yazılım güvenirliğini sağlamak için, müşteriden bilgi alınmasından teslime kadar bazı aktiviteler tanımlanmıştır.

- **Olabilirlik (fizibilite) ve gereksinimler:**
  - İşlevsel profili iyi belirleyin.
  - Hataları iyi tanımlayın ve sınıflandırın.
  - Müşterinin güvenirlik ihtiyaçlarını doğru tanımlayın.
  - Bütün organizasyonun yönetimini elinizde tutun.
- **Tasarım ve uygulama:**
  - Bileşenler arasında güvenirliği belirleyin.
  - İşlevsel kaynaklara odaklanın.
  - Hatalı kodlamaları görün.
  - Güvenirliği ölçmeye çalışın.
- **Sistem testi:**
  - Testleri yönetin ve gözlemleyin.
  - İhtiyaç duyulan ek testleri belirleyin.
  - Test edilen nesnelerin güvenirliğini ölçün.
- **Teslim ve bakım:**
  - Kullanıma geçiş sürecine doğru karar verin.
  - Güvenirliği gözlemleyin ve kontrol altında tutun.
  - Müşteri memnuniyetini izleyin.

  Hocanın vurgusu: Güvenirlik için yalnız bir model uygulamak yetmez; güvenirliği sağlamak için süreci de iyi yönetmek gerekir.

16. CK Metrikleri

Hoca son olarak yazılım mühendisliğinden bilinen bir konuyu hatırlattı: Nesneye yönelik yazılımlar için hangi özellikler önemlidir, diğer yazılımlara göre hangi özelliklere daha çok dikkat edilmelidir ve bunlar nasıl ölçülür? CK metrikleri adını Chidamber ve Kemerer'den alır; bu iki uzmanın çalıştığı ölçüm kümesidir. Slaytlarda kümedeki ölçümler ve bunların kaliteyle ilişkisi üç sayfada verildi.

16.1 WMC: Sınıfın Ağırlıklı Metot Sayısı

WMC (Weighted Methods per Class), bir sınıfın bütün metotlarının karmaşıklıklarının toplamıdır. Sınıfın geliştirilmesine ve bakımına ne kadar zaman harcanacağı konusunda fikir verir.

- Metot sayısı çok olan sınıflar çocuk sınıflarda daha çok etki bırakır, çünkü bütün metotlar türetilen sınıflarda da yer alır.
- Metot sayısı çok olan sınıfların uygulamaya özgü olma ihtimali daha yüksektir. Ağırlıklı metot sayısının fazla olması, belirli bir uygulama alanı (domain) için geliştirilmiş yazılımlarda daha çok görülür.
- Bu da yeniden kullanılabilirliği düşürür.

  Ders sonrası not: WMC anlatılırken 02:00:11'de "sınıf sayısı çok olan sınıflar" dendi. Doğrusu "metot sayısı (ağırlıklı karmaşıklığı) çok olan sınıflar"dır.

16.2 DIT: Kalıtım Ağacının Derinliği

DIT (Depth of Inheritance Tree), sınıfın kalıtım ağacının köküne uzaklığıdır. Sınıf hiçbir sınıftan türetilmemişse DIT sıfırdır. Hoca çoklu kalıtım varsa bu değerin daha fazla olduğunu söyledi. Derin kalıtım ağaçları daha çok tasarım karmaşıklığı oluşturur.

> **Ders sonrası not:** Chidamber ve Kemerer'in tanımında çoklu kalıtımda DIT, köke giden en uzun yolun uzunluğudur. Yani çoklu kalıtım değeri kendiliğinden artırmaz; büyük olasılıkla hoca çoklu kalıtımda yolların uzayabileceğini kastetti.

16.3 NOC: Alt Sınıf Sayısı

NOC (Number of Children), sınıftan doğrudan türetilmiş alt sınıfların sayısıdır. Bir sınıfın çok fazla alt sınıfı varsa bu, kalıtımın yanlış kullanıldığını gösterebilir. Çok alt sınıfı olan sınıfların metotları daha çok test gerektirir. Yani NOC fazlaysa test bütçesi de artar.

16.4 CBO: Nesne Sınıfları Arasındaki Bağımlılık

CBO (Coupling Between Object Classes), sınıfın bağımlı olduğu sınıf sayısıdır. Bağımlılık (coupling), sınıflar arasındaki ilişkinin sıkı olması, çok fazla ilişki bulunmasıdır. Tanım şudur: Bir sınıfın özellikleri ya da metotları başka bir sınıfta kullanılıyorsa ve bu sınıflar arasında kalıtım yoksa, iki sınıf arasında bağımlılık vardır.

- Sınıflar arasındaki aşırı bağımlılık modüler tasarıma zarar verir. Bu, yapısal yaklaşımdaki modül yapısı için de geçerlidir.
- Yeniden kullanılabilirliği azaltır. Bir sınıf ne kadar bağımsızsa o kadar yeniden kullanılabilir. Yeniden kullanılabilir bir kütüphane parçası olmak için sınıfın başka sınıflarla bağımlılığı çok olmamalı, daha genelleştirilmiş olmalıdır. Hoca bunu öğrencilerin uygulama geliştirdikleri için mantıksal olarak da anlayabileceklerini söyledi.
- Değişime duyarlılık arttığı için böyle bir yazılımın bakımı zordur.
- Bağımlılık, tasarımın farklı parçalarının ne kadar karmaşık test edileceği hakkında da fikir verir. CBO fazlaysa test maliyeti artar; test daha özenli ve dikkatli yapılmalıdır.

16.5 RFC: Sınıfın Tetiklediği Metot Sayısı

RFC (Response For a Class), verilen sınıftan bir nesnenin metotları çağrıldığında bu nesnenin tetikleyebileceği bütün metotların sayısıdır. Yani belirli bir sınıftan bir nesnenin metodu çağrıldığında bu nesne hangi metotları da çağırıyorsa, onların sayısıdır.

Bu metrik sınıfın test maliyeti hakkında fikir verir. Bir mesajın çok sayıda metodu tetiklemesi, sınıfın testinin ve hata ayıklamasının zorlaşması demektir. Bu yüzden bu sayının çok artmaması gerekir. Bir sınıftan çok sayıda metot çağrılıyorsa sınıf karmaşıktır; RFC sınıf karmaşıklığının bir işaretçisidir.

16.6 LCOM: Metotlardaki Uyum Eksikliği

LCOM (Lack of Cohesion in Methods) metriğinin birden fazla tanımı vardır. Hocanın verdiği CK tanımı şöyledir: C_1 sınıfının $M_1, M_2, \ldots, M_n$ metotları olsun ve I_j, M_j metodunda kullanılan nitelik (değişken) kümesi olsun. LCOM, bu n kümenin kesişimlerinden oluşan ayrık kümelerin sayısıdır. Bu tanımın türevleri de oluşturulmuştur.

Temel fikir şudur: Uyumun (cohesion) yüksek olması istenir. Metrik "uyum eksikliği" olduğu için LCOM değerinin az olması istenir. LCOM metotların uyumluluğunu gösterir.

> **Ders sonrası not:** Hocanın verdiği tanım (ayrık nitelik kümelerinin sayısı) sonraki türev tanımlara yakındır. Chidamber ve Kemerer'in 1994 tanımında büyük olasılıkla şöyledir: Ortak nitelik kullanmayan metot çiftlerinin sayısı P, ortak nitelik kullanan çiftlerin sayısı Q ise $P > Q$ olduğunda $LCOM = P - Q$, değilse $LCOM = 0$. Hesap mantığı python ile denetlendi.

16.7 CK Metrikleri Özeti

Metrik	Tanım	Yüksek olursa
WMC	Sınıftaki metotların karmaşıklıkları toplamı	Geliştirme ve bakım süresi artar, alt sınıflara etkisi büyür, yeniden kullanılabilirlik düşer
DIT	Sınıfın kalıtım ağacı köküne uzaklığı	Tasarım karmaşıklığı artar
NOC	Doğrudan türetilen alt sınıf sayısı	Kalıtım yanlış kullanılmış olabilir, test bütçesi artar
CBO	Kalıtım dışı bağımlı olunan sınıf sayısı	Modülerlik ve yeniden kullanılabilirlik düşer, bakım ve test zorlaşır
RFC	Bir nesnenin metotları çağrıldığında tetiklenebilecek metot sayısı	Test ve hata ayıklama zorlaşır, sınıf karmaşıktır
LCOM	Metotların nitelik kümelerinden oluşan ayrık küme sayısı	Uyum düşüktür; düşük olması istenir

Hoca bu örnekle, nesneye yönelik yaklaşıma has yapısal özellikler için de sayısal ölçütler ve ölçme sistemleri olduğunu gösterdiğini söyledi. Bunlar öğrencilerin yazılım mühendisliği ve nesneye yönelik yaklaşım derslerinde gördükleri, hatta uyguladıkları değerlerdir.

17. Gereksinim Belgesi (SRS) ve IEEE 830

Ders konuları bittikten sonra hoca, proje için hazırlanacak gereksinim raporu (SRS, Software Requirements Specification) hakkındaki soruları aldı.

Soru: Gereksinim raporu için paylaşabileceğiniz örnek bir dosya var mı? **Cevap:** Hoca öğrencilerin yazılım mühendisliği dersinde zaten SRS raporu hazırladığını, IEEE 830 standardını o derste paylaştığını hatırlattı. Ardından formatın ve bir örneğin üzerinden hızlıca geçti.

17.1 IEEE 830 Formatı

IEEE 830, SRS belgesi için IEEE tarafından hazırlanmış bir standarttır. Hoca bunun aynı zamanda standartlara bir örnek olduğunu belirtti: Standartlar ürüne değil, gidiş yoluna, yani ürünün nasıl hazırlanması gerektiğine yöneliktir. IEEE 830, SRS belgesinin hangi başlıklar altında ve hangi içerikle hazırlanacağını gösterir. Orijinal format çok ayrıntılıdır; aynısını kullanmak zorunlu değildir. Önemli olan gereksinimlerin listelenmesidir.

1. **Giriş:** Programın amacı, belgenin hangi sürüme (release) ait olduğu, ürünün kapsamı ve hangi belgeleri içerdiği.
2. **Genel tanım:** Hangi işletim ortamında çalıştığı, hangi ürün fonksiyonlarını gerçekleştirdiği, varsa kullanıcı el kitabı, kabuller ve kısıtlar (kullanıcı kısıtları, geliştirme kısıtları).
3. **Dış ara birim gereksinimleri:** Kullanıcı, donanım, yazılım ve iletişim ara birimleri. Program çalışırken böyle bir ara birim gereksinimi var mı?
4. **Sistem özellikleri:** Fonksiyonel gereksinimler; Requirement 1, Requirement 2 diye, yazılımın gerçekleştirmesi gereken temel işlevlerin maddeler halinde yazılması.

5. **Fonksiyonel olmayan gereksinimler:** Performans gereksinimleri, güvenlik, kalite özellikleri ve iş (işletmecilik) kuralları.

Ardından ekler gelir.

> **Hocanın vurgusu:** SRS'te esas istenen 4. ve 5. bölümlerdir. İlk üç bölüm yalnız temel özellikleri verecek kadar özet yazılabilir.

Gereksinimler programın tamamı için baştan sona (1, 2, 3, …) numaralanabilir ya da modüllere göre gruplanabilir: "şu temel modülle ilişkili gereksinimler" gibi alt başlıklar açılır. Hoca modül bazında yazmayı daha derli toplu bulduğunu söyledi. Orta büyüklükteki bir yazılımda bunlar toplam yaklaşık 50 gereksinim eder.

17.2 Örnek SRS

Hoca daha önceki bir ders ödevinden bir SRS örneği gösterdi: "Web Accessible Alumni Database". Belge amaç, genel görünüş ve ürünün genel tanımıyla başlıyor; fonksiyonel gereksinimler dördüncü bölümde yer alıyor.

- Büyük yazılım alt işlevlerine göre modüllere ayrılıp her modül alt başlıklarıyla yazılabilir.
- Bu örnekte olduğu gibi işlevsel gereksinimler bir genel use case diyagramıyla (aktörler ve alt başlıklar) ve her modül için use case belgeleriyle de gösterilebilir. Use case belgesinde nasıl alt işlemler yazılıyorsa, burada da her modülün yaptığı işlemler en basit haliyle tek tek yazılır. Örnekteki işlemler arasında mezun sayfasına erişim, bilgi girişi ve yeni bir giriş yaratılması vardı.
- Ardından fonksiyonel olmayan gereksinimler gelir: kullanıcı ara birim gereksinimleri ve performans gereksinimleri, yani o uygulamaya has ara birim ve performans gereksinimleri.

Hoca örneklerin birebir kopyalanmamasını istedi. IEEE 830 genel formatını kullanmakta sakınca yoktur, ama başka örnekler de bulunabilir.

Örnek Sorular

Bu derste ekranda gösterilen bir soru ya da quiz yoktur. Aşağıdaki sorular derste sözlü sorulan sorulardır.

1. (derste sözlü, 00:09:28) Doğrulama (verification) nedir?

> **Çözüm:** Öğrencinin cevabı "doğru yolla oluşturma" idi. Geliştirme sürecinde doğru yolu izleyip izlemediğimizin kontrolüdür: Birim testinde algoritmanın doğru yolla oluşturulup oluşturulmadığına, bütünlük testinde gereksinimlerin tasarıma doğru aktarılıp aktarılmadığına bakılır.

2. (derste sözlü, 00:11:04) Geçerleme (validation) nedir?

> **Çözüm:** Öğrenci cevap vermedi; hoca cevapladı. Üretilen yazılım ürünü istenen ürün mü? Müşterinin istediği işlevler ve ürün spesifikasyonu yazılımda sağlanıyor mu?

3. (derste sözlü, 00:30:15) McCall'a göre yazılım kalite özelliklerine hangi açılardan bakılır?

> **Çözüm:** Hocanın cevabı: kullanıcı açısından, geliştirici açısından ve işletici açısından.

4. (derste sözlü, 00:31:10) Kullanıcı yazılımın iyi test edilip edilmediğini hangi özelliklere bakarak anlar? Bu sistem testiyle ilişkilendirilebilir mi?

> **Çözüm:** Fonksiyonel olmayan kalite özelliklerine bakarak: verimlilik, güvenlik, dayanıklılık (stres). Evet; sistem testi bu fonksiyonel olmayan kalite özelliklerinin test edilmesidir.

5. (derste sözlü, 00:31:49) Dayanıklılıktan kastedilen nedir?

> **Çözüm:** Öğrencinin hocanın onayladığı cevabı: Kullanıcı sayısı belirlenen sınırın üzerine çıktığında ya da sınır değerlerde sistemin aynı verimlilikle çalışmaya devam etmesi.

6. (derste sözlü, 00:37:28) Size bir yazılım verildi. Bunu hangi sırada test edersiniz?

> **Çözüm:** Test süreci adımlarının sırasıyla, yani geliştirme sürecine ters sırada: birim testi, bütünlük testi (ve regresyon testi), sistem testi, onaylama ve kabul testi (alfa, beta), ardından işlemsel kullanım.

7. (derste sözlü, 01:04:04) Sınır değer analizinde hatalar genel olarak nerede yoğunlaşır?

> **Çözüm:** Sınırlarda. Hatalar merkezden çok kenarlarda toplanır; örneğin kullanıcı sayısının maksimum seviyesine yakın bir değerle program denenir.

8. (derste sözlü, 01:21:19 ve 01:23:15) Bir yazılımın büyüklüğü nasıl ölçülür? Hangisi doğrudan, hangisi dolaylı ölçümdür?

> **Çözüm:** Doğrudan ölçüm: satır sayısı (LOC) ve bin satır (KLOC). Dolaylı ölçüm: işlev noktası (function point).

9. (derste sözlü, 01:24:02) İşlev noktası hangi değerlerden hesaplanır?

> **Çözüm:** Girdi sayısı, çıktı sayısı, dosya (veritabanı tablosu) sayısı ve ekran (ara birim) sayısı; her biri farklı ağırlıklıdır. Ham değer, kalite özellikleriyle ilgili 14 sorunun 0-5 arası puanlarından bulunan düzeltme faktörüyle düzeltilir.

Dersin Toparlaması

Dersin ilk yarısı test sürecinin tekrarıydı. V modelde doğrulama, geliştirme sürecinde doğru yolu izleyip izlemediğimizi; geçerleme ise doğru ürünü üretip üretmediğimizi kontrol eder. Test adımları birim testi, bütünlük testi, regresyon, sistem testi ve onaylama/kabul testi sırasıyla ilerler. Sistem testi fonksiyonel olmayan kalite özellikleriyle eşleşir. Saydam kutu ve kara kutu testleri birbirinin alternatifi değil, tamamlayıcısıdır.

İkinci yarıda yazılım ölçümü işlendi. Halstead metriği işleç ve işlenen sayılarından aritmetiksel karmaşıklığı, McCabe ölçüsü akış grafından bağımsız yol sayısını verir; McCabe değeri modül büyüklüğüne karar vermekte de kullanılır (10'u geçmesin). Büyüklük doğrudan satır sayısıyla, dolaylı olarak işlev noktasıyla ölçülür. Güvenirlik, geçmiş bilgilere ve teste dayalı bir başarı oranıdır ve istatistiksel modellerle tahmin edilir; ama güvenirlik için süreci de yönetmek gerekir. Nesneye yönelik yazılımlarda CK metrikleri (WMC, DIT, NOC, CBO, RFC, LCOM) tasarımın karmaşıklığı, yeniden kullanılabilirliği ve test maliyeti hakkında fikir verir. Son olarak IEEE 830 formatıyla SRS belgesinin nasıl hazırlanacağı gösterildi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.